

Sistemas Operativos 2026



Clase 8

U3 Interbloqueos (Deadlocks) II

Ing. Juan Manuel Calvo

Evitar bloqueos

- Un sistema operativo puede usar esto para detectar y evitar bloqueos.
- Cuando se solicita un recurso, comprobar si concederlo creará un ciclo.
- De ser así, retrasar la concesión de ese recurso; ejecutar otro proceso en su lugar.
- Esperar a que se liberen otros recursos (que se corten los arcos) antes de conceder el recurso.

Detección y recuperación

- Caso simple: un recurso de cada tipo
- Construir un grafo de recursos
- Verificar si existen ciclos en el grafo (los algoritmos para hacerlo son bien conocidos)

3: Detección y recuperación

- Cuando nos damos cuenta que ha ocurrido un interbloqueo, tomamos alguna acción para que el sistema comience a funcionar nuevamente

Detección de interbloqueos

- Aún si no podemos prevenir los interbloqueos, quizás podamos **detectarlos** sistemáticamente.
 - Además vamos a necesitar un esquema para la recuperación
- O sea:
 - algoritmos para detectar los bloqueos mutuos
 - Ej. usar la teoría de los grafos
 - Una manera de deshacer (undoing) lo hecho antes que ocurriera el bloqueo
 - Ej. Revertir la transacción ("Rollback").

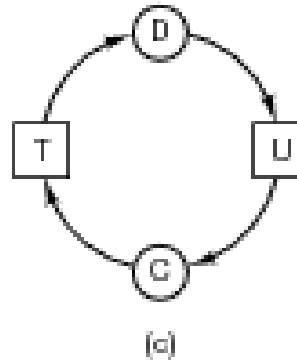


UNIVERSIDAD
DEL CEMA

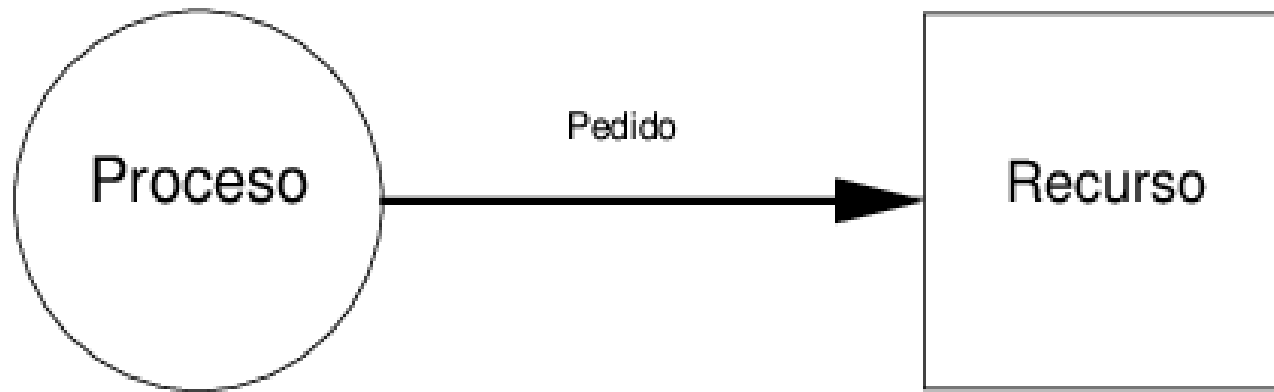
UCEMA

Modelado de un interbloqueo con grafos dirigidos

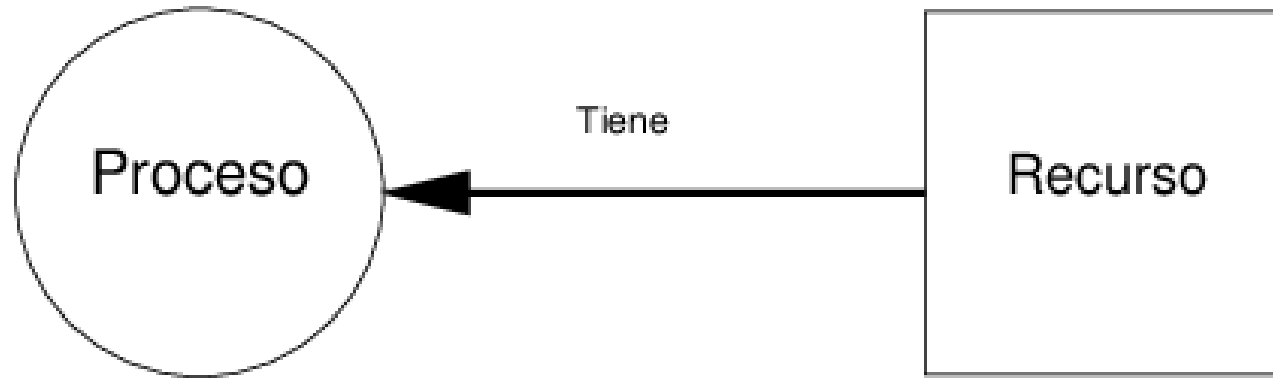
- Los círculos representan procesos, los cuadrados recursos.
- a) Un arco de un recurso a un proceso significa que el recurso está asignado al proceso.
- b) Un arco de un proceso a un recurso significa que el proceso está bloqueado esperando el recurso.



Pedido de un recurso

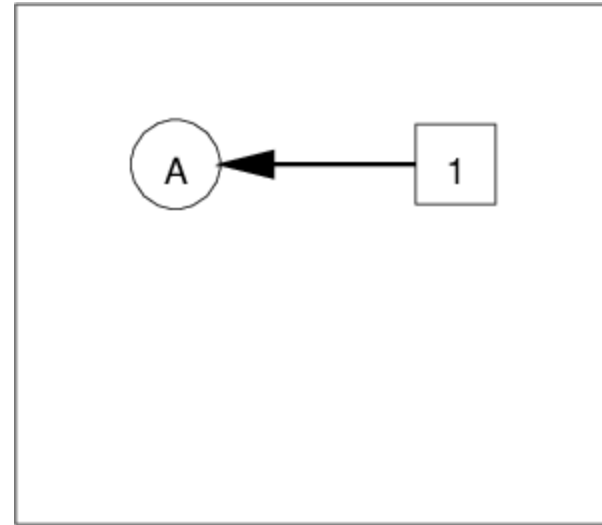


El proceso tiene un recurso



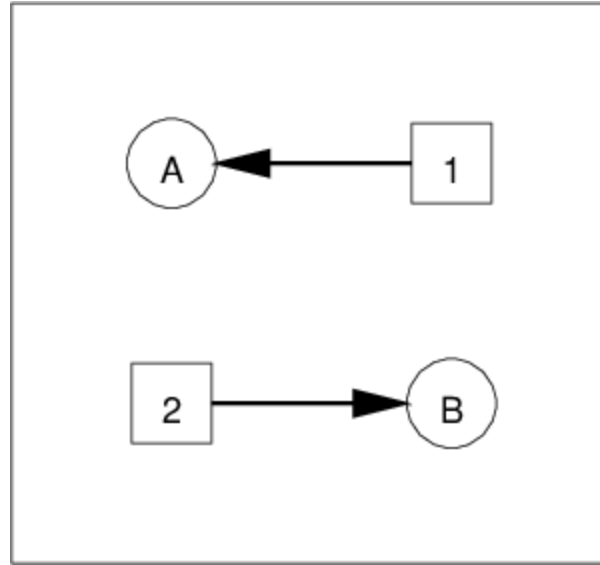
Código con bloqueo potencial (1)

```
typedef int semaphore;  
    semaphore resource1;  
    semaphore resource2;  
  
void processA(void) {  
    down(&resource1);  
    down(&resource2);  
    use both resources( );  
    up(&resource2);  
    up(&resource1);  
}  
  
void processB(void) {  
    down(&resource2);  
    down(&resource1);  
    use both resources( );  
    up(&resource1);  
    up(&resource2);  
}
```



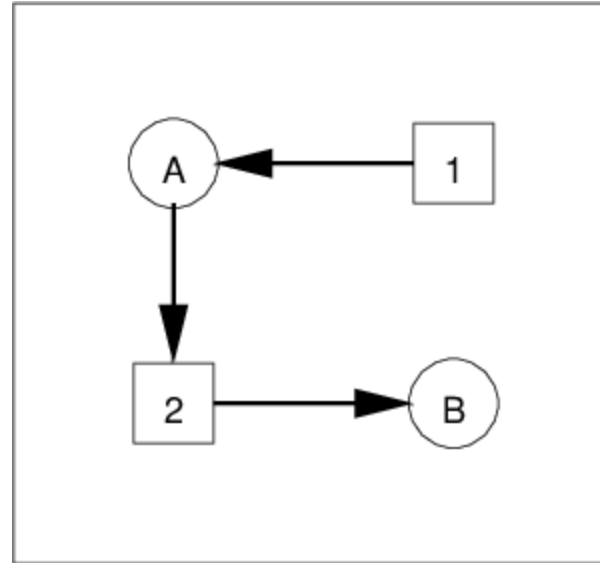
Código con bloqueo potencial (2)

```
typedef int semaphore;  
    semaphore resource1;  
    semaphore resource2;  
  
void processA(void) {  
    down(&resource1);  
    down(&resource2);  
    use both resources( );  
    up(&resource2);  
    up(&resource1);  
}  
  
void processB(void) {  
    down(&resource2);  
    down(&resource1);  
    use both resources( );  
    up(&resource1);  
    up(&resource2);  
}
```



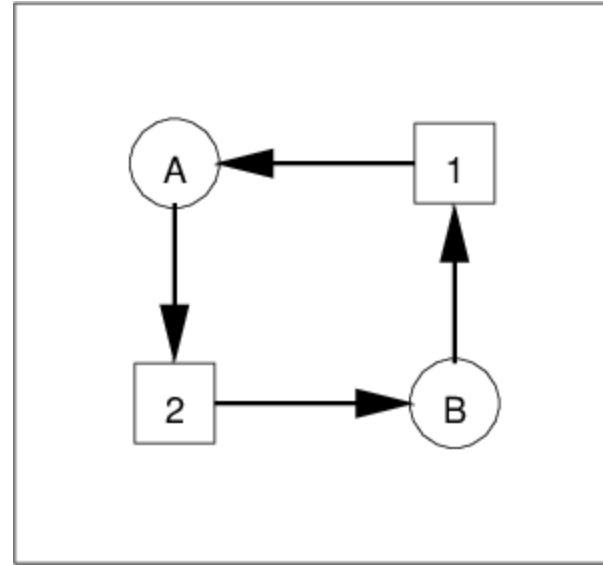
Código con bloqueo potencial (3)

```
typedef int semaphore;  
    semaphore resource1;  
    semaphore resource2;  
  
void processA(void) {  
    down(&resource1);  
    down(&resource2);  
    use both resources( );  
    up(&resource2);  
    up(&resource1);  
}  
  
void processB(void) {  
    down(&resource2);  
    down(&resource1);  
    use both resources( );  
    up(&resource1);  
    up(&resource2);  
}
```



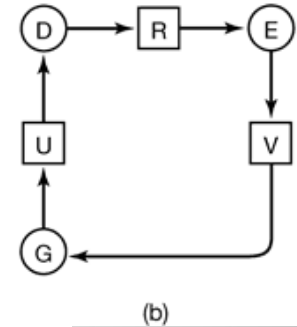
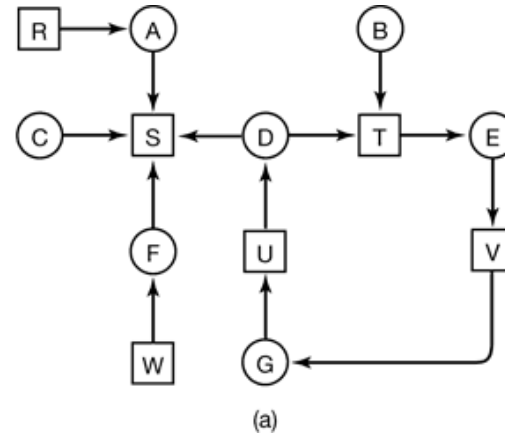
Código con bloqueo potencial (4)

```
typedef int semaphore;  
    semaphore resource1;  
    semaphore resource2;  
  
void processA(void) {  
    down(&resource1);  
    down(&resource2);  
    use both resources( );  
    up(&resource2);  
    up(&resource1);  
}  
  
void processB(void) {  
    down(&resource2);  
    down(&resource1);  
    use both resources( );  
    up(&resource1);  
    up(&resource2);  
}
```

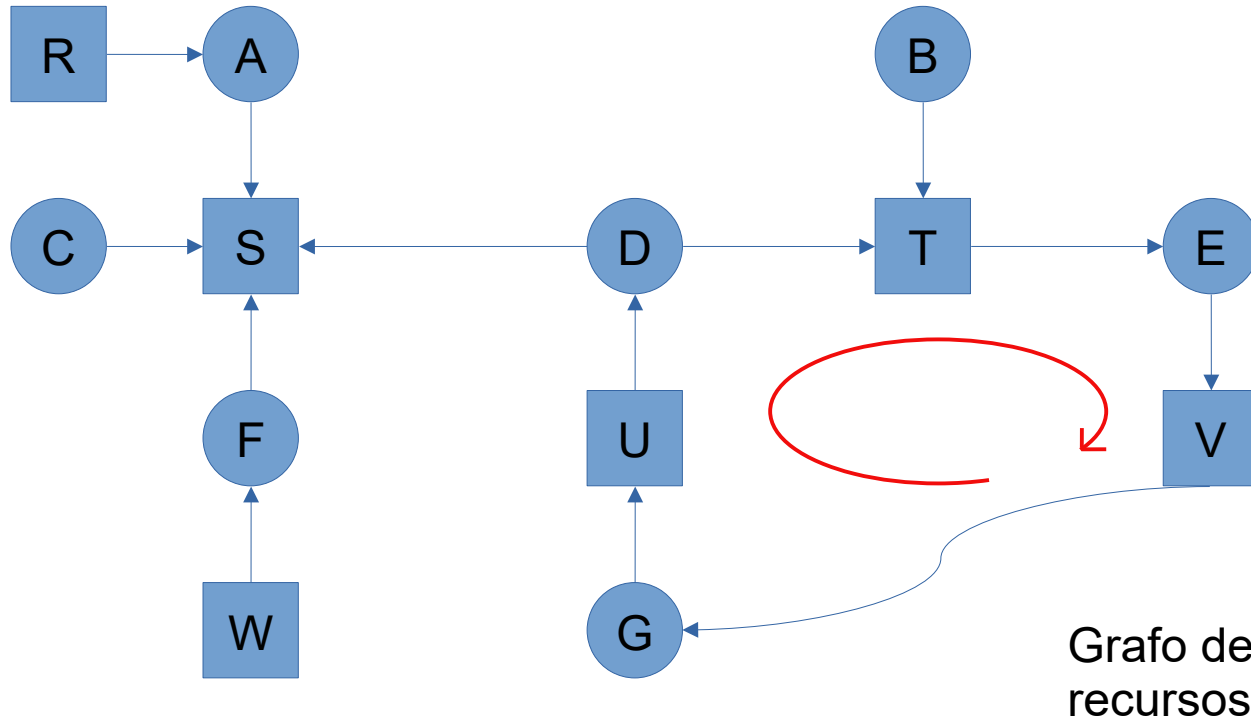


¿Hay buenos algoritmos de detección de bloqueos?

- Idea básica: representar la asignación de recursos como un grafo y buscar los lazos.
 - Sin lazos == no hay interbloqueos
 - Con lazos == hay interbloqueos
- Detección de lazos => problema típico de la teoría de grafos.
 - ¿puede ser hecho eficientemente?
 - ¿cuando probar?



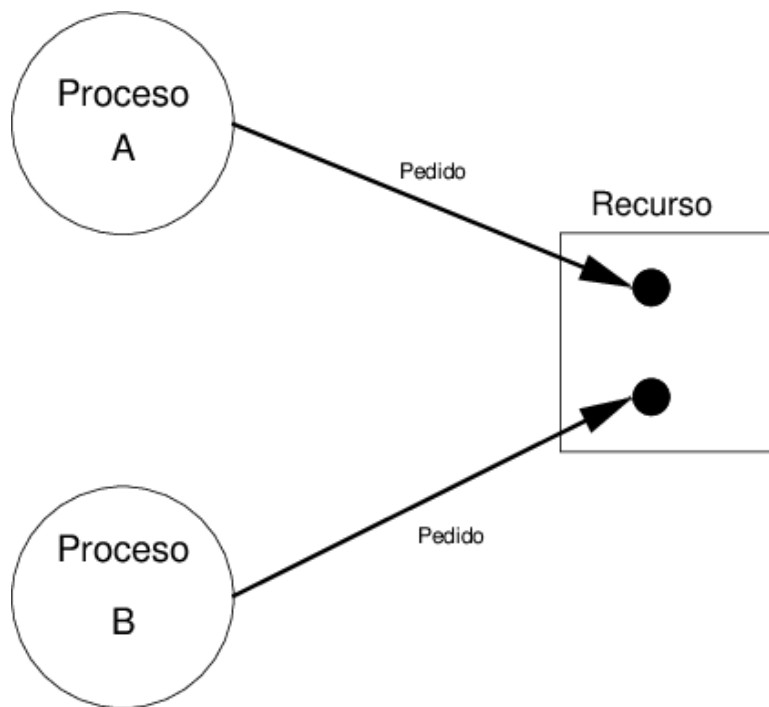
Deteccion de bloqueos con un recurso de cada tipo



Despues de la detección: recuperación de interbloqueos

- Quitar el recurso (preemption)
 - Sacarle el recurso a quien lo tiene
 - frecuentemente imposible
- Revertir la transacción (Rollback)
 - Puntos de chequeo periódicos guardando los estados
 - Volver al estado anterior antes de tomar el recurso
 - Usado en sistemas de bases de datos
- Matar los procesos
 - Crudo pero simple
 - Ir matando los procesos hasta que el lazo se rompa.

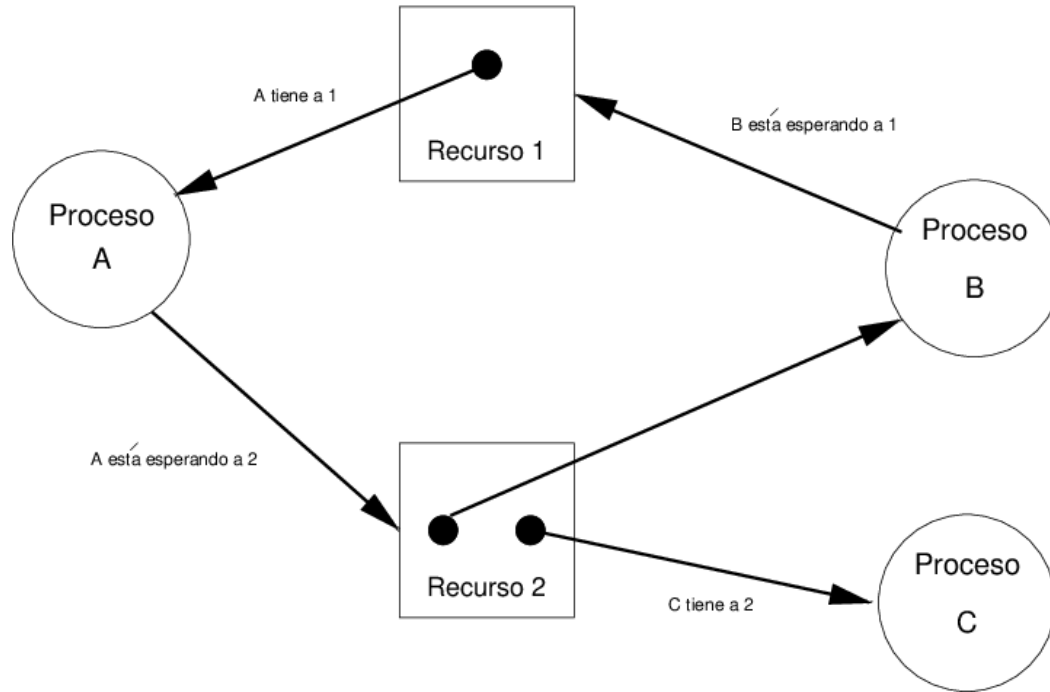
Recursos con varias instancias



UNIVERSIDAD
DEL CEMA

UCEMA

Recursos con varias instancias



Estrategias para enfrentar los interbloqueos

- Ignorar el problema: quizá no sea lo suficientemente grave/frecuente ✓
- Prevenirlo evitando una de las cuatro condiciones ✓
- Detectarlo y solucionarlo ✓
- Evitarlo mediante una asignación cuidadosa de recursos

Recuperación de un bloqueo

Varios métodos de recuperación (aunque ninguno es "atractivo"):

- 1) Retirar recursos
 - 1) Sacarle los recursos ya asignados
 - 2) Rollback
- 2) Matar procesos

Recuperación de un bloqueo: quitar recursos

- Seleccionar una víctima tratando de minimizar el costo
- Rollback: volver a un estado seguro, reiniciar el proceso desde ese estado
- Posibilidad de Inanición: elegir repetidamente el mismo proceso como víctima.

Recuperación de bloqueos: terminación de procesos

- Interrumpir todos los procesos bloqueados
 - De un proceso a la vez hasta que se elimine el ciclo de interbloqueo
- ¿En qué orden deberíamos elegir interrumpir?
 - Prioridad del proceso
 - Cuánto tiempo se ha corrido y cuánto tiempo falta para completarse
 - Recursos que ha utilizado el proceso
 - Recursos que el proceso necesita para terminar.
 - Cuántos procesos necesitarán ser terminados
 - ¿El proceso es interactivo o por batch?

Uso del algoritmo de detección

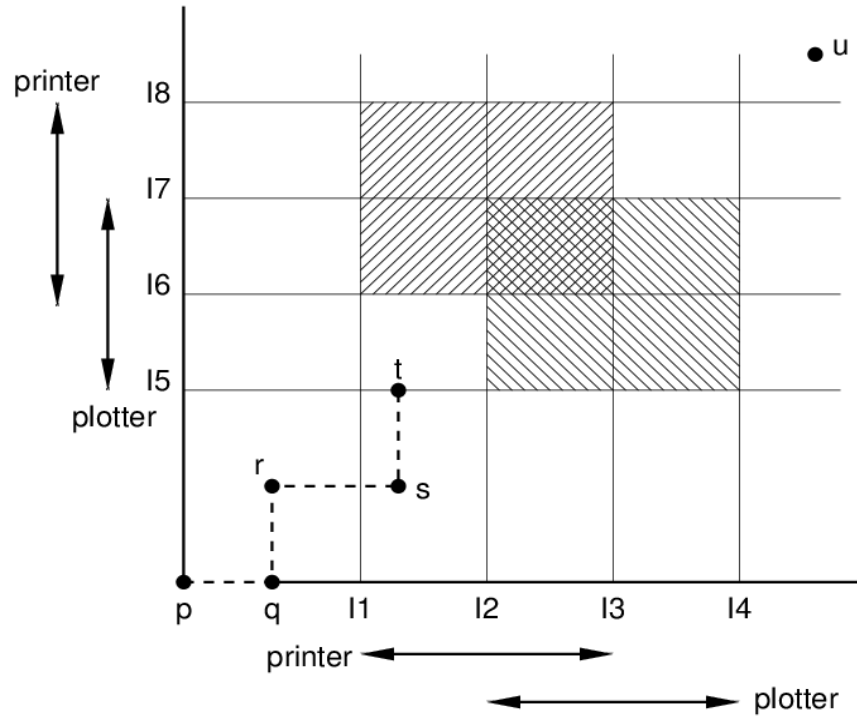
- Cuándo y con qué frecuencia invocar depende de:
 - ¿Cual es la frecuencia probable de la producción de bloqueos?
 - ¿Cuántos procesos deberán revertirse?
 - uno por cada ciclo disjunto
- Si el algoritmo de detección se invoca arbitrariamente, puede haber muchos ciclos en el gráfico de recursos y, por lo tanto, no podríamos saber cuál de los muchos procesos bloqueados "causó" el bloqueo.

4: Evitar el problema

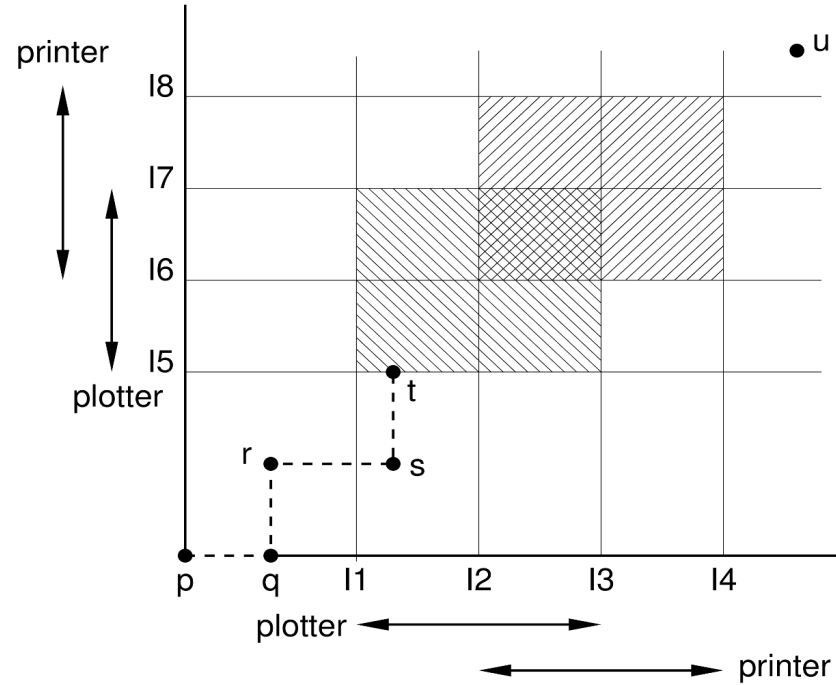
- Trayectorias de recursos
- Estados seguros e inseguros
- Algoritmo del banquero



Evitar los bloqueos: trayectorias de recursos



Aquí no hay posibilidad de bloqueos



Evitar los bloqueos

Requiere que el sistema tenga disponible a priori alguna información adicional.

- El modelo más simple y útil requiere que cada proceso declare el número máximo de recursos de cada tipo que puede necesitar.
- El algoritmo para evitar los bloqueos examina dinámicamente el estado de asignación de recursos para garantizar que nunca pueda haber una condición de espera circular.
- El estado de asignación de recursos se define por el número de recursos disponibles y asignados y las demandas máximas de los procesos.

Estados Seguros

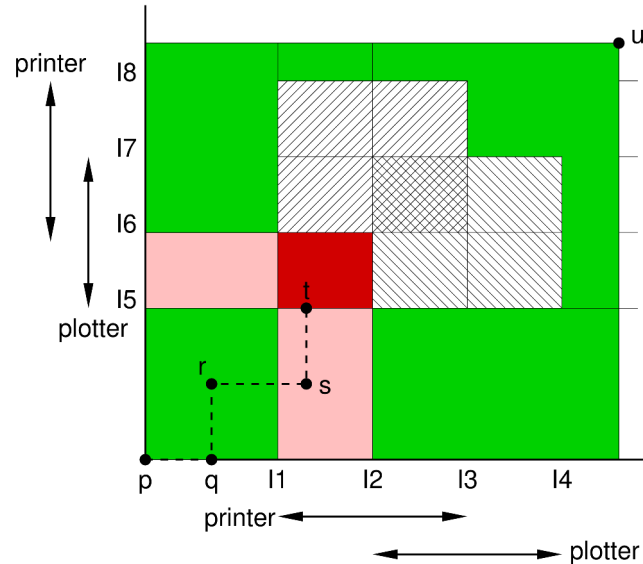
- Cuando un proceso solicita un recurso disponible, el sistema debe decidir si la inmediata asignación deja al sistema en un estado seguro.
- Se dice que **un estado es seguro si hay algún orden de programación en el que todos los procesos pueden ejecutarse hasta su finalización incluso si todos ellos solicitan su número máximo de recursos.**
 - Si las necesidades de recursos de P_i no están disponibles de inmediato, entonces P_i puede esperar hasta que todas las P_j hayan terminado
 - Cuando P_j finaliza, P_i puede obtener los recursos necesarios, ejecutarlos, devolver los recursos asignados y terminar
 - Cuando P_i termina, P_{i+1} puede obtener los recursos necesarios, y así sucesivamente.

Estados seguros e inseguros

- Un estado es seguro si no está bloqueado y existe un orden de programación en el que todos los procesos pueden completarse, incluso si solicitan todos sus recursos a la vez.
- Un estado inseguro no está bloqueado, pero no existe tal orden de programación.
- Un estado inseguro Incluso podría funcionar si un proceso libera recursos en el momento adecuado.

Estados seguros, inseguros y bloqueados

- Si un sistema está en estado seguro $\Rightarrow$ no son posibles los bloqueos
- Si un sistema está en un estado inseguro $\Rightarrow$ posibilidad de bloqueo
- Evitar $\Rightarrow$ asegúrese de que un sistema nunca entrará en un estado inseguro



Estados seguros e inseguros

a)

	Tien e	Má x
A	3	9
B	2	4
C	2	7

Libres:3

b)

	Tien e	Má x
A	3	9
B	4	4
C	2	7

Libre:1

c)

	Tien e	Má x
A	3	9
B	-	-
C	2	7

Libres:5

d)

	Tiene	Máx
A	3	9
B	-	-
C	7	7

Libre:0

e)

	Tiene	Máx
A	3	9
B	-	-
C	-	-

Libres:7

Demostración que el estado a) es seguro

Estados seguros e inseguros (2)

a)

	Tien e	Má x
A	3	9
B	2	4
C	2	7

Libres:3

b)

	Tien e	Má x
A	4	9
B	2	4
C	2	7

Libres:2

c)

	Tien e	Má x
A	4	9
B	4	4
C	2	7

Libre:0

d)

	Tiene	Máx
A	4	9
B	-	-
C	2	7

Libre:4

Demostración que el estado b) es inseguro



UNIVERSIDAD
DEL CEMA
UCEMA

Algoritmos para evitar los bloqueos

- Instancia única de un tipo de recurso
 - Utilice un gráfico de asignación de recursos
- Varias instancias de un tipo de recurso
 - Utilice el algoritmo del banquero



Algoritmo del banquero

- Aplicable en el caso de múltiples instancias
- Cada proceso debe reclamar a priori el máximo uso de cada recurso
- Cuando un proceso solicita un recurso, puede que tenga que esperar
- Una vez que un proceso obtiene todos sus recursos, debe devolverlos en un período de tiempo finito.

Algoritmo del banquero

- Inconvenientes:
 - Rara vez sabe por adelantado cuántos recursos necesitarán los procesos
 - el número de procesos cambia a medida que pasa el tiempo
 - Un recurso una vez disponible puede desaparecer

Resumen de enfoques para la prevención de los bloqueos

Condición	Enfoque
Exclusión mutua	Spooling
Retención y espera	Pedir todos los recursos de una sola vez
Quitarle los recursos al que los tiene	
Espera circular	Ordenar los recursos numericamente

Livelock (bloqueo activo)

- El bloqueo activo se produce cuando un hilo intenta continuamente una acción que falla todas las veces.
 - Puede ser evitado repitiendo la acción después de una demora aleatoria.

